Understanding RealTOA

A Telegram Bot that automates the mileage tracking for SAF Transport Operators (TO), hence becoming a 'Real TO Assistant (RealTOA)'

1 INTRODUCTION

1.1 CONTEXT

In the Singapore Armed Forces (SAF), Transport Operators (TOs) are required to log the mileage accumulated during each mission. This record is important because upon reaching 4000km, a TO is eligible to convert his military driving license to a civilian one, free of cost.

Therefore, **accurate, timely and complete mileage records of the TOs are important**.

1.2 CURRENT WORKFLOW

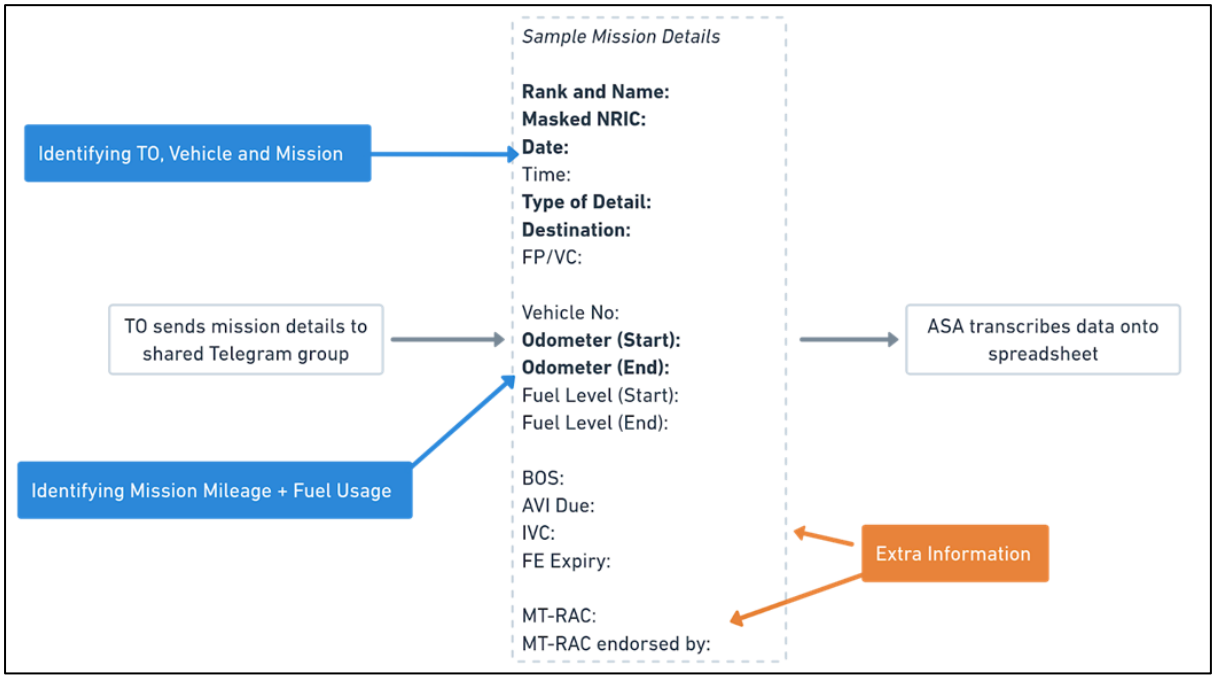


Figure 1: Current Workflow

Currently, the TO:

1. Uploads mission information into a shared Telegram group
2. Admin Support Assistants (ASAs) then transcribes information onto a spreadsheet

From this workflow, fellow TOs and I have noticed some potential and ongoing failure points, observed in Table 1 below.

Failure	Impact
Messages missed or overlooked by ASA	Entry is not recorded → lost mileage for mission
Incorrect mission details sent	Inaccurate odometer logs → inaccurate mileage totals (worse if it is lower than actual)
No TO visibility of spreadsheet	TOs cannot verify their own records and mileage without requesting the ASAs for access
Numerous TOs → numerous mission details sent everyday	Potential delays in transcription + repetitive, tiresome for ASAs.

Table 1: Identified Failure Points

1.3 Objectives

Hence, this project aims to:

- **Reduce Human Error** in mileage recording for both the TOs and ASAs.
- **Improve the efficiency and accuracy of updating** and maintaining the mileage records
- Create a more **transparent and reliable** mileage recording system
- **Automate the mileage recording system**

2 IDEATION

2.1 DESIGN REQUIREMENTS

To automate the current workflow, the initial approach was to use Google Forms, due to its straightforward integration with Google Sheets for automated data recording. However, observations in the node show that TOs are **less responsive** to submitting information through structured forms. This is mainly due to 2 reasons.

Firstly, the form-based submission requires TOs to fill up multiple input fields, increasing perceived effort, reducing efficiency. In contrast, TOs are used to sending information quickly through Telegram, often by adapting previously sent messages. Introducing a new, structured form disrupts this established workflow, and increases the effort required to submit information.

Secondly, Google Forms does not provide immediate confirmation that submitted data has been appended to the spreadsheet, unless the sender (TO) manually verifies the spreadsheet. This lack of feedback reduces user confidence and discourages adoption.

Based on these observations, the following design requirements were established, as observed in Table 2 below.

Problems Identified	Design Requirements
Low responsive to Google Forms	The system should **integrate directly** with TOs' existing telegram group
	The system should **require minimal changes to the current message-posting behaviour.**
	The system should **not require** TOs to use **any additional software or external tools**.
Lack of feedback	The system should **notify TOs** once their information has been appended in the spreadsheet.

Table 2: Design Guidelines

2.2 Design Proposals

To address the identified requirements, two primary design proposals have been ideated, developed and evaluated.

2.2.1 Proposal 1 – Integrating 'Structured-Forms' into Telegram

This approach replicates a structured form-based input system (like Google Forms) within Telegram through a bot interface.

2.2.1.1 Workflow Description

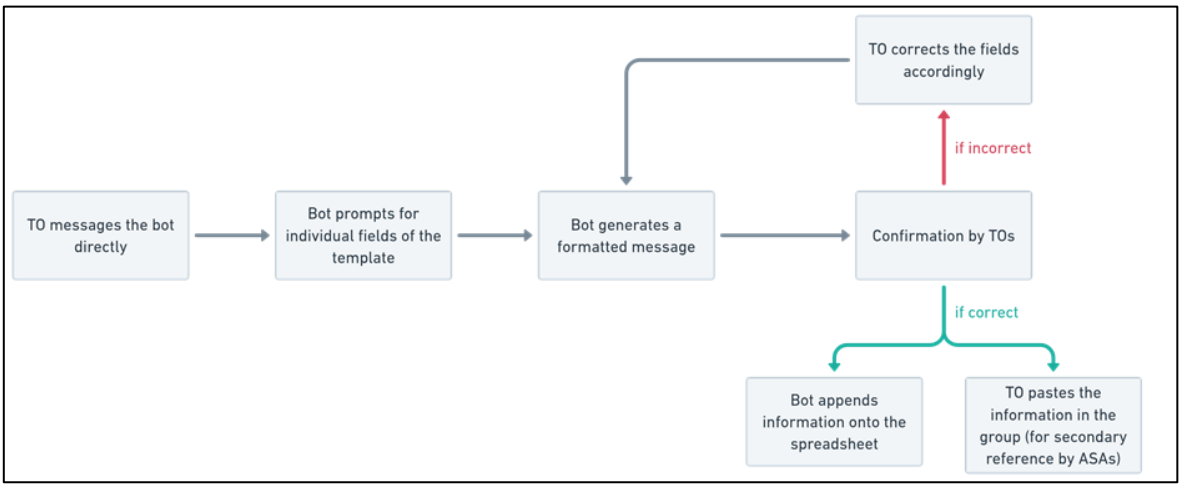


Figure 2: Proposal 1's Workflow

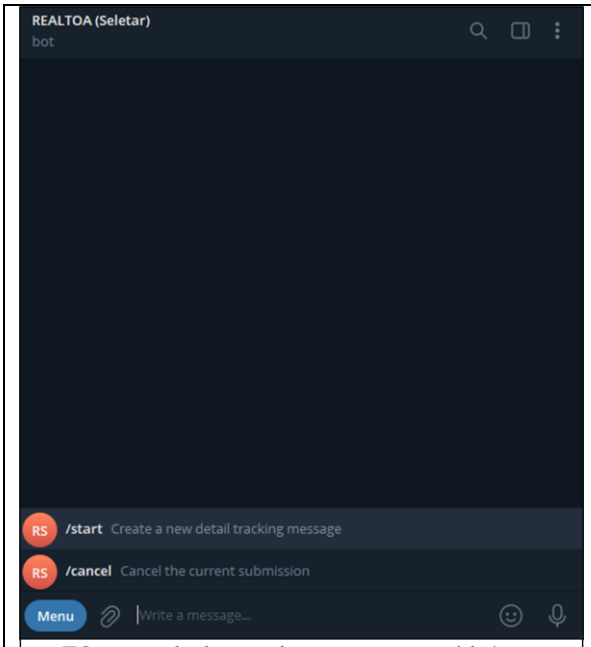
The workflow operates in the following manner:

1. The TO initiates the process by messaging the bot
2. The bot sequentially prompts the TOs for required fields in the mission template (shown in Figure 1: Current Workflow).
3. Relevant personal information is pre-filled to reduce redundant effort.
4. The bot compiles all completed fields into a formatted message.
    a. Upon confirmation by the TO, this information is appended to the spreadsheet, where it is verified by the ASAs.
    b. The TO sends this message in the Telegram group (for secondary verification and reference for ASAs).

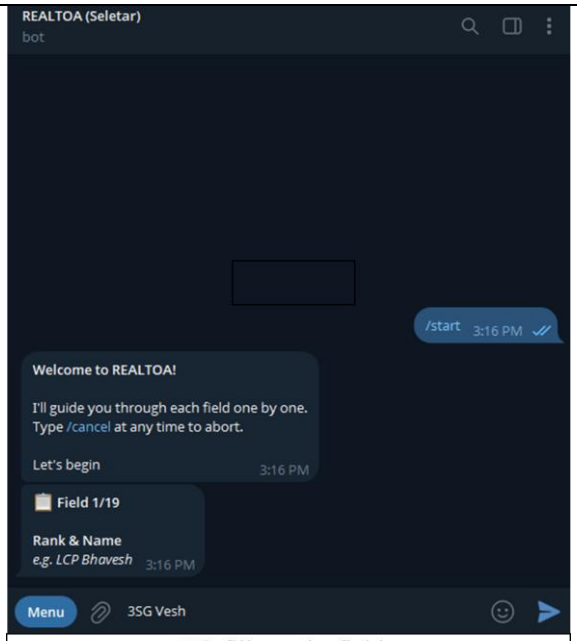
2.2.1.2 Design Rationale

Proposal 1 aims to preserve the benefits of a structured data input process (found in Google Forms), while maintaining TO's comfort by making it Telegram-native.

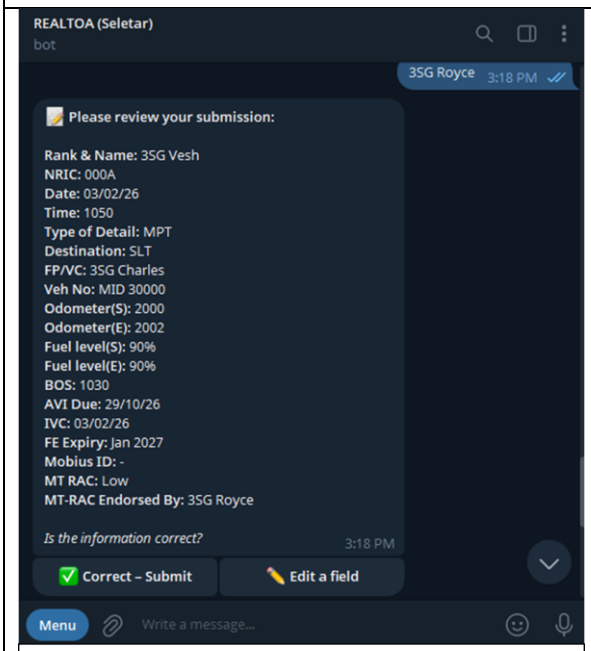
2.2.1.3 Proof of Concept



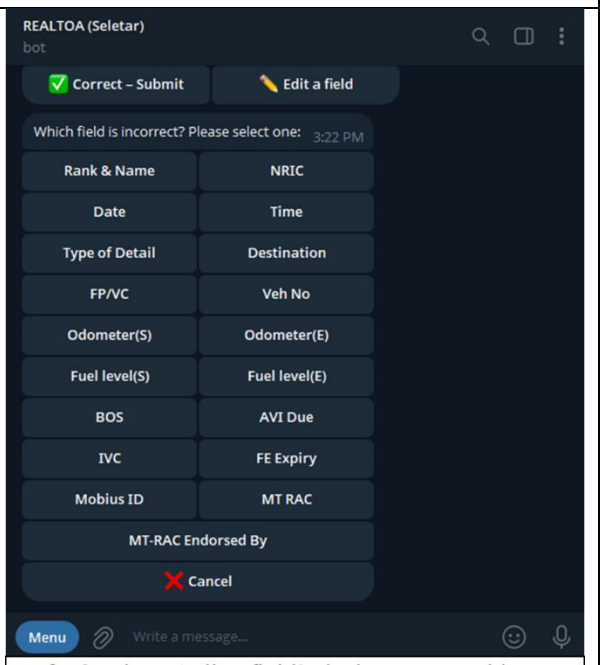
TO opens the bot, and starts a convo with /start



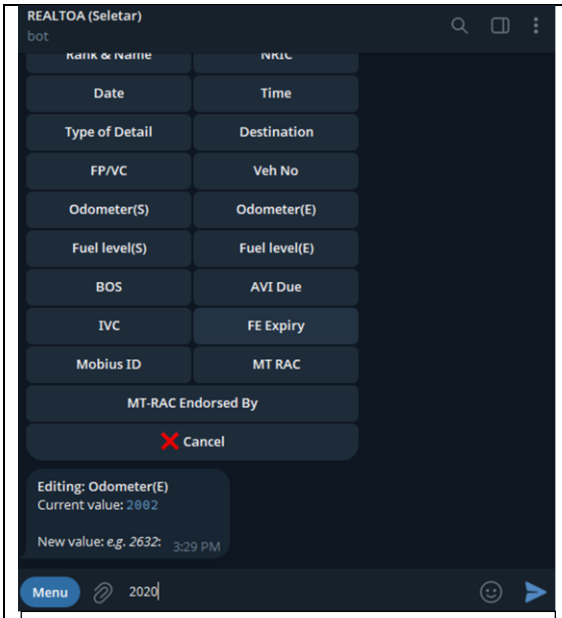
TO fills up the fields



After finishing the fields, the bot prompts TO to check sent information thus far

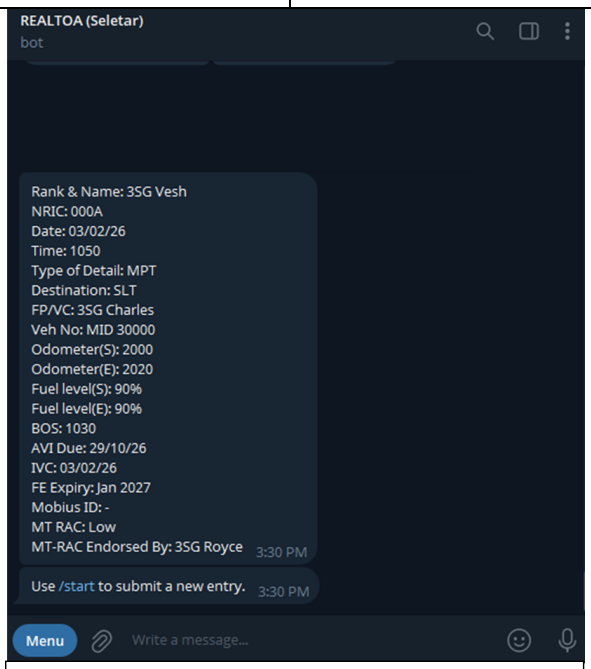


If TO selects 'edit a field', the bot prompts him to select which field to update.



Bot shows what field is being edited, its current value, and proposed format of the new value.

Bot sends the updated message for confirmation.



If the TO selects yes, this message is sent for the purpose of 'copy-pasting' to the Telegram group with ASAs

Table 3: Screenshots of Proposal 1

2.2.1.4 Evaluating Proposal 1

Advantages	Limitations
Pre-filled fields → reduces TO effort → faster uploads	Numerous sequential prompts → increase interaction time → perceive bot to be bothersome → reduced engagement
No additional software required (everything native in Telegram) → less behavioural friction → higher likelihood of compliance	Introduction of a bot still involves behavioural friction
Confirmation Loop → TOs forced to verify their records → Accurate mileage tracking	Manual copying and pasting of messages from the bot to the telegram group introduces redundant effort.
Logic for appending data to spreadsheet is simple and straightforward.	Requires continuous uptime for bot to be available for 24/7 messaging and appending.
	API costs for appending to spreadsheet

Table 4: Evaluation of Proposal 1

2.2.2 Proposal 2 – Passive, Parsing Telegram Bot

Proposal 2 is also a telegram bot, but relies on passive automation. It retains existing communication means, while the bot/system extracts relevant information from the Telegram Messages automatically.

2.2.2.1 Workflow Description

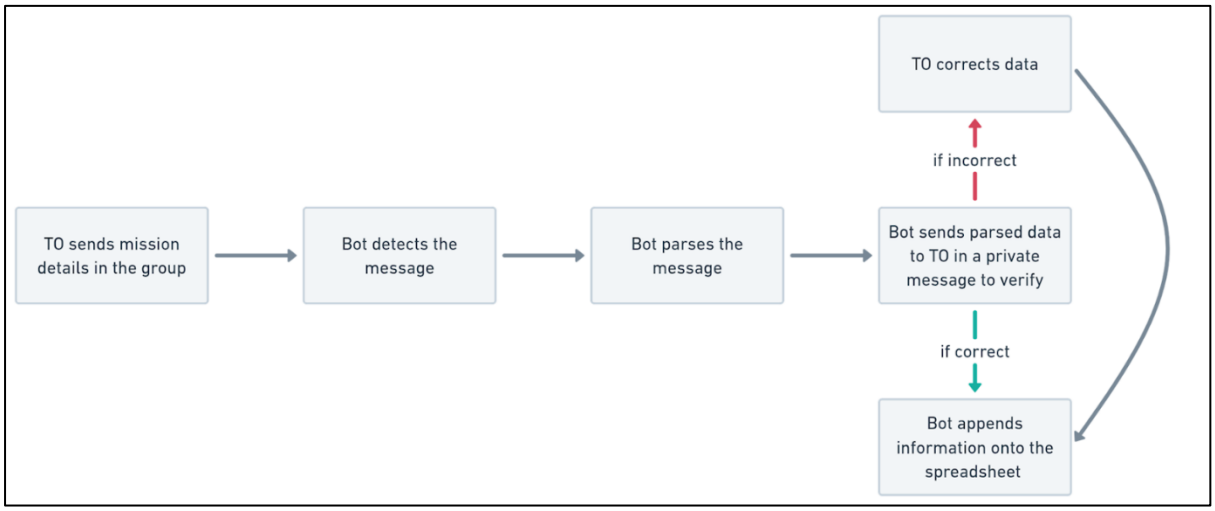


Figure 3: Proposal 2's Workflow

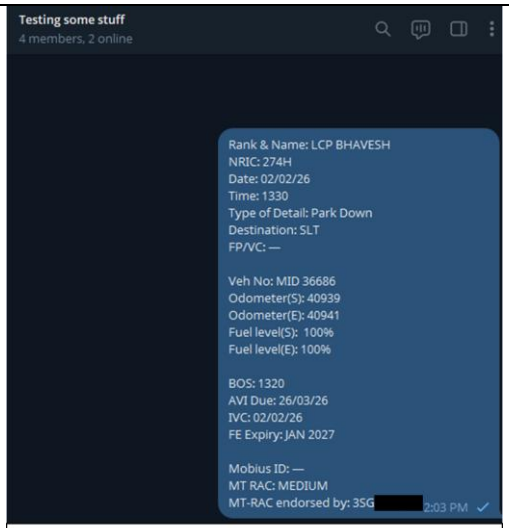
The workflow operates in the following manner:

1. The TO sends mission details in the Telegram group normally.
2. The bot detects the message.
3. The bot extracts key information from the message using *regex parsing*.
4. The bot messages the sender TO privately to confirm extracted information.
    a. Upon confirmation, this information is appended to spreadsheet.
    b. If inaccurate, the TO is prompted to correct relevant fields in the bot, and is expected to edit information in the Telegram group.

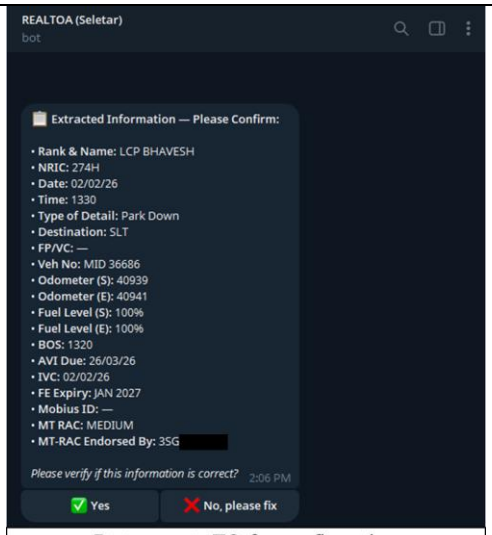
2.2.2.2 Design Rationale

This Telegram Bot prioritises TO's behavioural compatibility by minimising user interaction with the bot, and retaining existing communication patterns

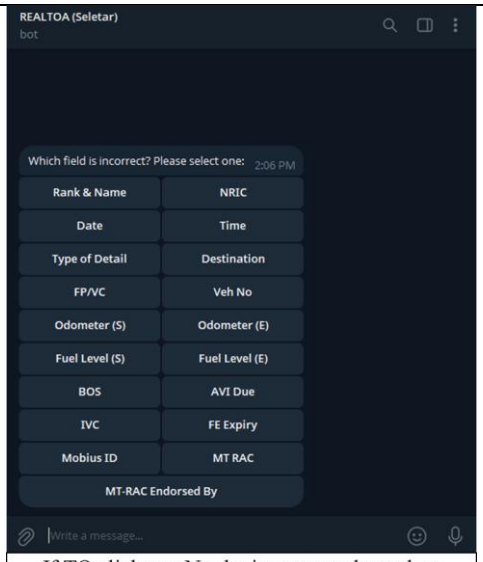
2.2.2.3 Proof of Concept



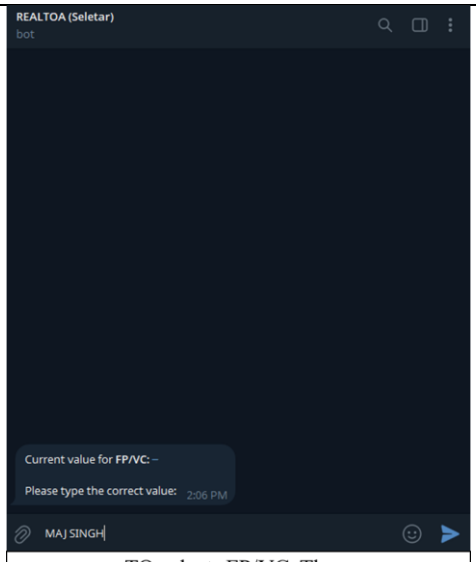
TO sends mission details in the group



Bot requests TO for confirmation

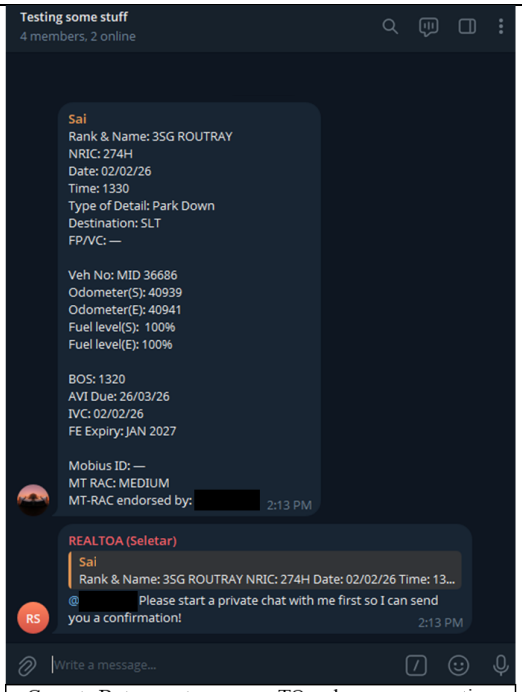


If TO clicks on No, he is prompted to select which field.



TO selects FP/VC. Then,
TO is prompted to update this field.

Message updates. TO prompted to confirm message again.



Caveat: Bot cannot message TO unless a conversation has started.
Hence, bot prompts TO in the group to start a convo.

Table 5: Screenshots of Proposal 2

2.2.2.4 Evaluating Proposal 2

Advantages (proposal 2)	Limitations (proposal 1)	Limitations (proposal 2)
Minimised interaction with the bot, the bot prompts rather than the TO prompts	~~Introduction of a bot may still involve some behavioural friction~~	Regex Parsing logic may be difficult to code and implement due to numerous, possible edge cases.
Less prompts and questions by the bot	~~Numerous prompts still yet to answer → may perceive bot to be annoying~~	
Less work done by TOs (information remains in the telegram group primarily)	~~Copy, pasting from the bot to the telegram group can be mundane → skipped → secondary reference for ASAs missing~~	
	Requires continuous uptime for bot to be available for 24/7 messaging and appending.	
	API costs for appending to spreadsheet	

Table 6: Evaluation of Proposal 2

2.3 Comparative Analysis

Proposals 1 and 2 have been evaluated against certain criteria, as show in

Criteria	Proposal 1	Proposal 2
Behavioural Compatibility	Moderate	High
User Effort	Moderate	Low
Technical Difficulty	Low	High
Data Accuracy	High	Moderate

Table 7: Cross Evaluation

Comparing both proposals, Proposal 2 evidently aligns stronger with user behaviour. It minimally disrupts the existing workflow, requires lower user effort, hence increasing behavioural compatibility. These are important factors in ensuring long-term adoption and effectiveness.

However, these benefits come at a cost of increased technical complexity, especially in developing the parsing logic and error-handling systems, that would improve and maintain high data accuracy. Nevertheless, these technical complexities can be mitigated through repeated rounds of testing and continuous improvements to the system's logic.

Thus, given the benefits of increased user compliance, ease of use, better system integration, outweighing technical complexities, **Proposal 2 (Passive, Parsing Telegram Bot) is the preferred solution.**

3 TESTING

To test the proposals, there are 2 main categories to consider:

1. Behavioural Testing (determining if the TOs like the solution)
2. Technical Testing (determining if the system can handle the expected load and function reliably)

The testing will be conducted in the following manner:

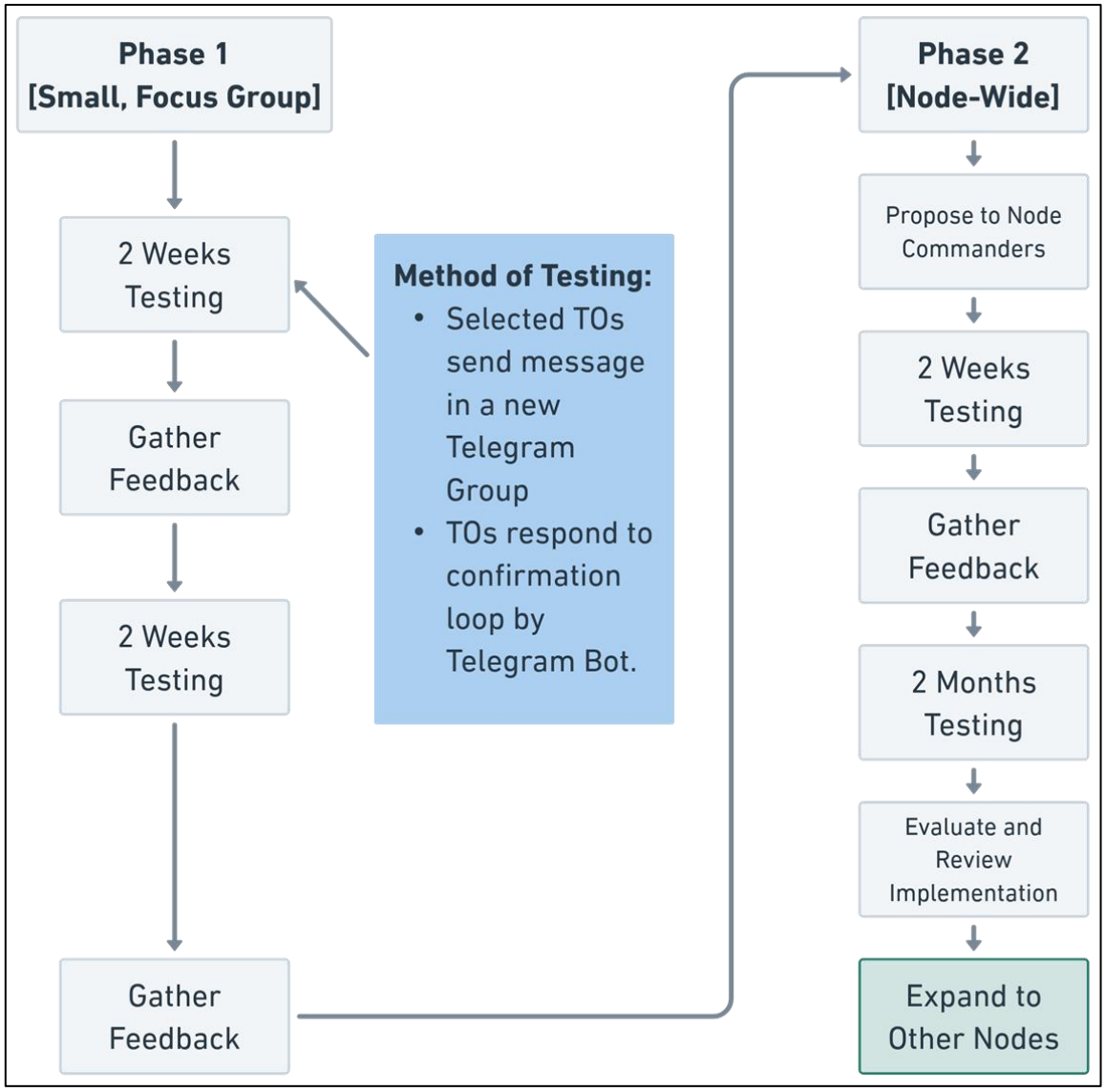


Figure 4: Proposed Testing Flowchart

3.1 Testing Timeline

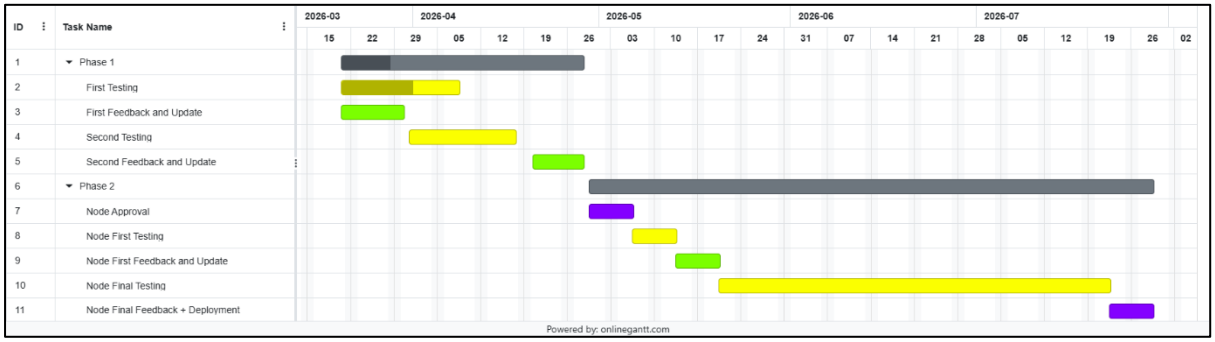


Figure 5: Gantt Chart for Testing

Task Name		Start Date	End Date
Phase 1	First Testing	02/03/2026	16/03/2026
	First Feedback and Update (current)	17/03/2026	22/03/2026
	Second Testing	23/03/2026	06/04/2026
	Second Feedback and Update	07/04/2026	12/04/2026
Phase 2	Node Approval	13/04/2026	26/04/2026
	Node First Testing	27/04/2026	11/05/2026
	Node First Feedback and Update	12/05/2026	17/05/2026
	Node Final Testing	18/05/2026	17/07/2026
	Node Final Update + Deployment	18/07/2026	27/07/2026

Table 8: Important Deadlines

3.2 CURRENT SHORTCOMINGS

3.2.1 Current System Shortcomings

Current Limitation	Description	Future Fix
No local storage	If the spreadsheet append fails or API quota empties out prematurely, there is no intermediate storage before appending to a spreadsheet again (once quota replenishes). Thus, data is lost unless the user resubmits manually again.	Connect to a local database (MongoDB, SQLite, etc) for intermediate data storage, when 'spreadsheet appending system' is down. Once API quota replenishes, the script should automatically transfer data from the local database to the spreadsheet.
Appending to Google Sheets is inactive	Since no rate-limiting has been implemented, there is a risk of unintended high-frequency writes that can use up the free quota quickly. There is no code in place that prevents a TO from sending duplicate entries.	Implement a per-user cooldown Introduce a duplicate-entry detection by checking key fields (Vehicle No., Odometer (S))
No format checks for user inputs of mission details.	User-inputs are currently accepted without any format checks. 1. 'Date, AVI due date, FE expiry, IVC' fields are not validated against a DD/MM/YY pattern. This can conflict when appending to the spreadsheet. 2. Fuel levels are not limited to just percentages (some TOs input them as decimals instead)	Create a date converter, which recognise common messaging date formats, and converts them to DD/MM/YY for appending.

Table 9: Current System Shortcomings and Fixes

3.2.2 Infrastructure Limitations

Infrastructural Limitations	Description	Future Fix
The bot is not available 24/7	The bot uses polling. Thus, it is only reachable while the laptop is powered on and the script is running.	Deploy the bot to a cloud-based service, or into a server. Change from polling to webhooks.
No Bot crash resolution	If the bot crashes, there is no back-up version of the bot for users to use.	Create a *process manager* to automatically restart the bot if it crashes.

Table 10: Current Infrastructure Limitations and Fixes